SVKM's NMIMS

Mukesh Patel School of Technology Management & Engineering (Mumbai Campus)

Computer Engineering Department (B Tech CSE/CSBS Sem IV/BTI Sem VIII/MBA.Tech-IV)

Database Management System

Project Report

Program	Btech CSBS	
Semester	IV	
Name of the Project:	Vehicle Rental Management System	
Details of Project Members		
Batch	Roll No.	Name
B2	E042	Shreyash Nikam
B2	E044	Swapnil Pal
B2	E050	Soumya Ranasingh
Date of Submission: 26/02/2025		

Contribution of each project Members:

Roll No.	Name:	Contribution
E042	Shreyash Nikam	SQL Queries,Report
E044	Swapnil Pal	Backend Development
E050	Soumya Ranasingh	Frontend Development, Report

Github link of your project: https://github.com/ShreyashNikam17/Vehicle-Rental-System

Project Report

CAR RENTAL MANAGEMNET SYSTEM

by

Shreyash Nikam, Roll number: E042

Swapnil Pal, Roll number: E044

Soumya Subhankar Ranasingh, Roll number: E050

Course: DBMS

AY: 2024-25

Table of Contents

I.Storyline

Imagine a city where every engine hum, every booking confirmation, and every returning customer adds a note to the ever-evolving symphony of urban mobility. In a world rapidly shifting towards on-demand convenience, the Car Rental System emerges—not just as software, but as a dynamic data ecosystem, designed to power every journey, from spontaneous weekend getaways to everyday urban commutes.

At its core, this project is about building a robust database system that becomes the operational backbone of a rental business. The system intricately records each critical detail: user registrations, vehicle availability, rental transactions, return logs, and admin updates. Each entry is more than a number—it's a snapshot of movement, preference, and behavior.

The project unfolds in two powerful layers. First, the Java application integrated with MySQL via JDBC ensures a seamless interaction between users and data. Admins can add, update, or delete car entries; clients can create accounts, browse available vehicles, rent cars, and view their history—all through a user-friendly GUI. The system isn't just a set of menus; it's an intelligent assistant that tracks everything from car usage trends to user activity, instantly reflecting changes in the database.

Underneath this interaction lies SQL—used rigorously to extract, update, and manage data across multiple tables. Structured queries ensure data integrity and speed. Whether it's verifying a password, checking if a car is already rented, or fetching rental history based on a client's ID, the system responds with precision and efficiency.

The second layer is the strategic insight this data unlocks. Through this system, a rental business isn't just executing transactions—it's learning. Admins can track the most rented cars, identify peak booking hours, detect idle inventory, and even foresee maintenance cycles based on usage patterns. The system evolves from a database to a business brain.

But this is more than a technical build—it's empowerment. For a customer, it's a smooth rental experience, from login to key handover. For an admin, it's complete visibility into the fleet and client base. And for the business, it's a system that translates data into direction.

Ultimately, the Car Rental System DBMS project is a fusion of relational database architecture and real-world logic. It captures motion, records intention, and responds to demand—one transaction at a time. Behind every rented car is a story, and behind every story, a line of code that made it possible.

II. Components of Database Design

User

Attributes:

user_id (Primary Key)
first_name
last_name
email
phone_number
password
type (0 = Client, 1 = Admin)

Overview:

In your system, all registered individuals (admins and clients) are stored in the users table. The type attribute differentiates between Admin and Client roles. Every user has a unique user_id, and this table forms the backbone of the entire system.

Client

Derived from users where type = 0
Primary Key: user_id (also a Foreign Key from users)

Overview:

Clients are general users who can rent cars through the GUI. In your Java GUI, clients can:
Create accounts
View available cars
Rent cars
View their rental history

Admin

Derived from users where type = 1
Primary Key: user_id (also a Foreign Key from users)

Overview:

Admins are users with elevated privileges who can:
Add new cars
Update or delete cars
View all client rental histories
Manage the overall fleet

Car

Attributes:

car_id (Primary Key)
brand
model
color
year
price (hourly rate)
availability (1 = available, 0 = rented)

Overview:

This table maintains the inventory of cars available in the system. Each car is uniquely identified by car_id and contains detailed specifications including brand, model, and availability status. Admins manage this data through GUI operations like Add Car, Delete Car, and Update Car.

Rent

Attributes:

rent_id (Primary Key)
user_id (Foreign Key → users.user_id)
car_id (Foreign Key → cars.car_id)
date (rental date/time)
hours (number of hours rented)
total (calculated amount)
status (0 = ongoing, 1 = completed)
Overview:
This table logs each rental transaction. When a client rents a car via the GUI, a new record is inserted here. It tracks which client rented which car, for how long, and what total amount was paid. It's also used for showing client-specific rental history.

Relationships and Their Role in the Project

User – Client / Admin

Type: One-to-One
Participation:
User: Optional (a user can be created without a role initially)
Client/Admin: Mandatory (must map to an existing user)
Project Link:
In the system, roles are selected during account creation in the GUI. Admins and clients share the same base structure (users table) but operate with different GUI options and access rights.

Client – Rent

Type: One-to-Many
Participation:
Client: Optional (a client might not have rented yet)
Rent: Mandatory (each rental is done by a client)
Project Link:
Clients can rent multiple cars via the GUI. Their rentals are recorded in the rents table. Admins can view all rentals; clients can view only their own.

Car – Rent

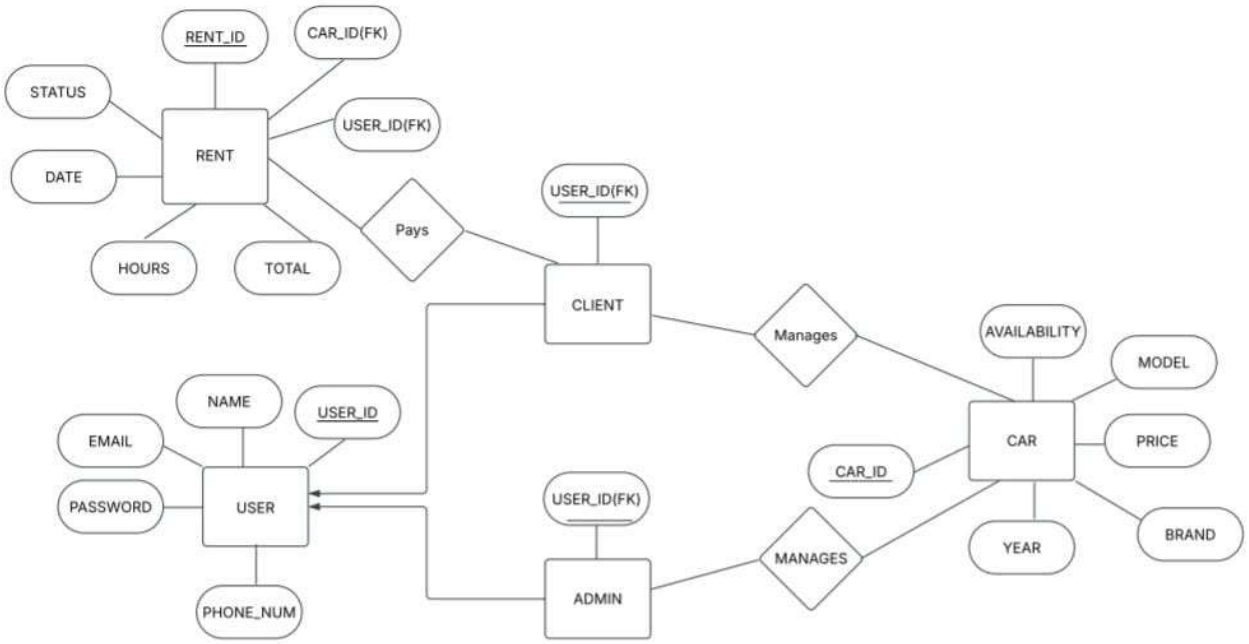
Type: One-to-Many
Participation:
Car: Mandatory (every rental must be linked to an existing car)
Rent: Mandatory
Project Link:
When a car is rented, the car's availability is set to 0 (unavailable). The GUI triggers both an INSERT into rents and an UPDATE in cars.

Admin – Car

Type: One-to-Many

Participation:
Admin: Mandatory (each car should be registered by an admin)
Car: Optional (a car may exist before being assigned)
Project Link:
In the backend code (AddNewCar.java), an admin logs in and inserts new cars. You can expand this by linking cars to the admin_id for tracking who added what (a good future enhancement).

III. Entity Relationship Diagram


RENT_ID
CAR_ID(FK)
STATUS
USER_ID(FK)
RENT
DATE
Pays
USER_ID(FK)
HOURS
TOTAL
CLIENT
Manages
AVAILABILITY
MODEL
NAME
USER_ID
EMAIL
CAR
PRICE
CAR_ID
USER_ID(FK)
PASSWORD
USER
MANAGES
YEAR
BRAND
ADMIN
PHONE_NUM

IV. Relational Model

1. USERS

Table Name: USERS
Attributes:

- user_id (Primary Key)
- name
- email
- password
- phone_num

Schema:

```
USERS(
  user_id   INT         PRIMARY KEY,
  name      VARCHAR(100),
  email     VARCHAR(150) UNIQUE,
  password  VARCHAR(255),
  phone_num VARCHAR(15)  UNIQUE
);
```

2. CLIENT

Table Name: CLIENT
Attributes:

- user_id (Primary Key & Foreign Key → USERS.user_id)

Schema:

CLIENT(

user_id INT PRIMARY KEY REFERENCES USERS(user_id)

);

3. ADMIN

Table Name: ADMIN

Attributes:

user_id (Primary Key & Foreign Key → USERS.user_id)

Schema:

ADMIN(

user_id INT PRIMARY KEY REFERENCES USERS(user_id)

);

4. CAR

Table Name: CAR
Attributes:

- car_id (Primary Key)
- model
- brand

- year
- price
- availability

Schema:

```
CAR(
  car_id       INT          PRIMARY KEY,
  model        VARCHAR(100),
  brand        VARCHAR(100),
  year         INT,
  price        DECIMAL(10, 2),
  availability BOOLEAN
);
```

5. RENT

Table Name: RENT
Attributes:

- rent_id (Primary Key)
- user_id (Foreign Key → USERS.user_id)
- car_id (Foreign Key → CAR.car_id)
- status
- date
- hours
- total

Schema:

RENT(

rent_id INT PRIMARY KEY,

user_id INT FOREIGN KEY REFERENCES USERS(user_id),

car_id INT FOREIGN KEY REFERENCES CAR(car_id),

status VARCHAR(50),

date DATE,

hours INT,

total DECIMAL(10, 2)

);

6. ADMIN_CAR_MANAGEMENT

Table Name: ADMIN_CAR_MANAGEMENT
Attributes:

- admin_id (Foreign Key → ADMIN.user_id)
- car_id (Foreign Key → CAR.car_id)

Schema:

ADMIN_CAR_MANAGEMENT(

admin_id INT FOREIGN KEY REFERENCES ADMIN(user_id),

car_id INT FOREIGN KEY REFERENCES CAR(car_id),

PRIMARY KEY (admin_id, car_id)

);

V.Normalization

What is Normalization?

Normalization is a systematic process used to organize data in a way that reduces redundancy, eliminates data anomalies, and ensures data integrity.

It involves dividing large, complex tables into smaller, related ones and defining relationships between them using keys (primary and foreign).

There are 3 types of Normal Form:

1. **1st Normal Form(1NF)**
   - **Definition:** First Normal Form (1NF) is a property of a relational database table that ensures all the data in each column is atomic, meaning each field contains only one value, and that there are no repeating groups or duplicate rows. This form ensures that the table structure is organized, readable, and suitable for further normalization.
   - **Where is it used:**
     - Tables like USERS, CAR, RENT, etc., have clearly defined atomic fields: `name`, email, phone_num, model, brand, availability, etc.
     - No multivalued fields like rented_car1, rented_car2 etc.
2. **2nd Normal Form(2NF)**
   - **Definition:** Second Normal Form (2NF) is achieved when a table is already in First Normal Form (1NF) and all non-key attributes are fully functionally dependent on the entire primary key. This means there should be no partial dependency, where a non-key column depends on only a part of a composite primary key. If the table has a single-column primary key, it automatically satisfies this condition if it's in 1NF.
   - **Where is it used:**
     - Tables like CLIENT, ADMIN, CAR, USERS have single primary keys.
     - Even in the ADMIN_CAR_MANAGEMENT table, which has a composite key (admin_id, car_id), there are no non-prime attributes—only the keys exist.
3. **3rd Normal Form(3NF)**
   - **Definition**: Third Normal Form (3NF) is achieved when a table is already in Second Normal Form (2NF) and all non-key attributes are directly dependent on the primary key only, and not on other non-key attributes.There should be no transitive dependency, where a non-key column depends on another non-key column instead of directly on the primary key.
   - **Where is it used**:
     - In USERS, email, name, password, phone_num all depend only on user_id.
     - In RENT, status, date, hours, total depend only on rent_id.
     - RENT.total might seem like a derivable field (from price × hours), but as it's stored directly, it's not violating 3NF.

VI.SQL Queries

1)

```sql
CREATE DATABASE IF NOT EXISTS carrentalsystem DEFAULT CHARACTER SET utf8mb4 COLLATE utf8mb4_general_ci;
USE carrentalsystem;
```

2)

```sql
DROP TABLE IF EXISTS cars;
CREATE TABLE IF NOT EXISTS cars (
  ID int(11) NOT NULL,
  Brand text NOT NULL,
  Model text NOT NULL,
  Color text NOT NULL,
  Year int(11) NOT NULL,
  Price double NOT NULL,
  Available int(11) NOT NULL
);
```

3)

```sql
INSERT INTO cars (ID, Brand, Model, Color, Year, Price, Available) VALUES
(0, 'Brand 0', 'Model 0', 'Color 0', 2020, 1000, 0),
(1, 'Brand 1', 'Model 1', 'Color 1', 2021, 1100, 0),
(2, 'Brand 2', 'Model 2', 'Color 2', 2022, 1200, 0),
(3, 'Brand 3', 'Model 3', 'Color 3', 2023, 1300, 0),
(4, 'Brand 4', 'Model 4', 'Color 4', 2024, 1400, 0),
(5, 'Brand 5', 'Model 5', 'Color 5', 2025, 1500, 0),
(6, 'Brand 6', 'Model 6', 'Color 6', 2026, 1600, 0),
(7, 'Brand 7', 'Model 7', 'Color 7', 2027, 1700, 0),
(8, 'Brand 8', 'Model 8', 'Color 8', 2028, 1800, 2),
(9, 'Brand 8', 'Model 8', 'Color 8', 2028, 1800, 0),
(10, 'Brand 9', 'Model 9', 'Color 9', 2029, 1900, 0);
```

4)

```sql
DROP TABLE IF EXISTS rents;
CREATE TABLE IF NOT EXISTS rents (
  ID int(11) NOT NULL,
  User int(11) NOT NULL,
  Car int(11) NOT NULL,
  DateTime text NOT NULL,
  Hours int(11) NOT NULL,
  Total double NOT NULL,
  Status int(11) NOT NULL
);
```

5)

```sql
INSERT INTO rents (ID, User, Car, DateTime, Hours, Total, Status) VALUES
(0, 2, 7, '2023-22-12 23:59', 2, 3400, 0),
(1, 2, 0, '2023-22-12 23:59', 7, 7000, 0),
(2, 2, 2, '2023-23-12 00:00', 3, 3600, 0),
(3, 2, 3, '2023-23-12 00:16', 1, 1300, 0),
(4, 2, 5, '2023-23-12 00:16', 2, 3000, 0),
(5, 2, 5, '2023-23-12 00:16', 5, 7500, 0),
(6, 2, 9, '2023-23-12 00:16', 8, 14400, 0),
(7, 2, 7, '2023-23-12 00:16', 7, 11900, 0),
(8, 2, 5, '2023-23-12 00:16', 1, 1500, 0);
```

6)

```sql
DROP TABLE IF EXISTS users;
CREATE TABLE IF NOT EXISTS users (
  ID int(11) NOT NULL,
  FirstName text NOT NULL,
  LastName text NOT NULL,
  Email text NOT NULL,
  PhoneNumber text NOT NULL,
  Password text NOT NULL,
  Type int(11) NOT NULL
);
```

7)

```sql
INSERT INTO users (ID, FirstName, LastName, Email, PhoneNumber, Password, Type) VALUES
(0, 'Admin', '0', 'admin', '0000', '0000', 1),
(1, 'Admin', '2', 'admin2', '222222', '1234', 1),
(2, 'Client', '1', 'client', '111111', '1111', 0),
(3, 'Client', '2', 'client2@crs.com', '222222', '2222', 0),
(4, 'Client', '3', 'client3@crs.com', '333333', '3333', 0),
(5, 'Client', '4', 'client4@crs.com', '444444', '4444', 0),
(6, 'Client', '5', 'client5@crs.com', '555555', '5555', 0),
(7, 'Client', '6', 'client6@crs.com', '666666', '6666', 0),
(8, 'Client', '7', 'client7@crs.com', '777777', '7777', 0),
(9, 'Client', '8', 'client8@crs.com', '888888', '8888', 0),
(10, 'Client', '9', 'client9@crs.com', '999999', '9999', 0);
```

```sql
SELECT
  u.FirstName, u.LastName, u.Email,
  c.Brand, c.Model, c.Color,
  r.DateTime, r.Hours, r.Total
FROM rents r
JOIN users u ON r.User = u.ID
JOIN cars c ON r.Car = c.ID;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

FirstName	LastName	Email	Brand	Model	Color	DateTime	Hours	Total
Admin	2	admin2	Brand 0	Model 0	Color 0	2023-12-22 23:59	7	7000
Admin	2	admin2	Brand 0	Model 0	Color 0	2023-12-22 00:00	3	3600

8)

9)

```
15
16      -- listing non admin users
17 •    SELECT * FROM users
18      WHERE Type = 0;
19
20
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ID	FirstName	LastName	Email	PhoneNumber	Password	Type
3	Client	1	client1	111111	1111	0
4	Client	2	client2@crs.com	222222	2222	0
5	Client	3	client3@crs.com	333333	3333	0
6	Client	4	client4@crs.com	444444	4444	0
7	Client	5	client5@crs.com	555555	5555	0

Result 4 | cars 5 | users 6 ×

10)

```
20      -- counting total cars
21 •    SELECT COUNT(*) AS TotalCars
22      FROM cars;
23
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

TotalCars
5

Result 7 | cars 8 | users 9 | Result 10 ×

11)

```
24      -- total rentals done by each user
25      SELECT u.FirstName, u.LastName, COUNT(r.ID) AS Rentals
26      FROM rents r
27      JOIN users u ON r.User = u.ID
28      GROUP BY r.User;
```

```
-- Car models rented more than once
SELECT c.Model, COUNT(r.ID) AS TimesRented
FROM rents r
JOIN cars c ON r.Car = c.ID
GROUP BY r.Car
HAVING COUNT(r.ID) > 1;
```

12)

13)

```
38      -- average
39  •   SELECT AVG(Hours) AS AvgHours
40      FROM rents;
41
42
```

Result Grid | Filter Rows: | Export

AvgHours
3.7778

14)

```
42      -- total revenue
43  •   SELECT c.Model, SUM(r.Total) AS Revenue
44      FROM rents r
45      JOIN cars c ON r.Car = c.ID
46      GROUP BY r.Car;
47
```

Result Grid | Filter Rows: | Export: | Wrap Cell

Model	Revenue
Model 0	10600

15)

```
47
48      -- latest 5 rentals
49      SELECT *
50      FROM rents
51      ORDER BY DateTime DESC
52      LIMIT 5;
53
54
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ID	User	Car	DateTime	Hours	Total	Status
3	2	5	2023-12-23 00:16	1	1300	0
4	2	5	2023-12-23 00:16	2	3000	0
5	2	5	2023-12-23 00:16	3	7500	0
6	2	5	2023-12-23 00:16	8	14400	0
7	2	7	2023-12-23 00:16	7	11900	0

Result 49 | cars 50 | users 51 | Result 52 | Result 53 | Result 54 | Result 55

16)

```
54      -- users who rented cars in 2025
55      SELECT DISTINCT u.FirstName, u.Email
56      FROM users u
57      JOIN rents r ON u.ID = r.User
58      WHERE r.DateTime LIKE '2023%';
59
```

Result Grid | Filter Rows: | Export: | Wra

FirstName	Email
Admin	admin2

17)

```
63 •    SELECT ID, FirstName
64      FROM users
65      WHERE ID IN (
66          SELECT User
67          FROM rents
68          GROUP BY User
69          HAVING SUM(Total) > 5000
70      );
71
```

Result Grid | Filter Rows: | Export:

ID	FirstName
2	Admin

18)

```
73      -- Rentals longer than 5 hours
74      SELECT *
75      FROM rents
76      WHERE Hours > 5;
77
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

	ID	User	Car	DateTime	Hours	Total	Status
▶	1	2	0	2023-12-22 23:59	7	7000	0
	6	2	5	2023-12-23 00:16	8	14400	0
	7	2	7	2023-12-23 00:16	7	11900	0

```
-- unique users
SELECT c.Model, COUNT(DISTINCT r.User) AS UniqueRenters
FROM rents r
JOIN cars c ON r.Car = c.ID
GROUP BY r.Car;
```

lt Grid | Filter Rows: | Export: | Wrap Cell Content:

lodel	UniqueRenters
odel 0	1

19)

20)

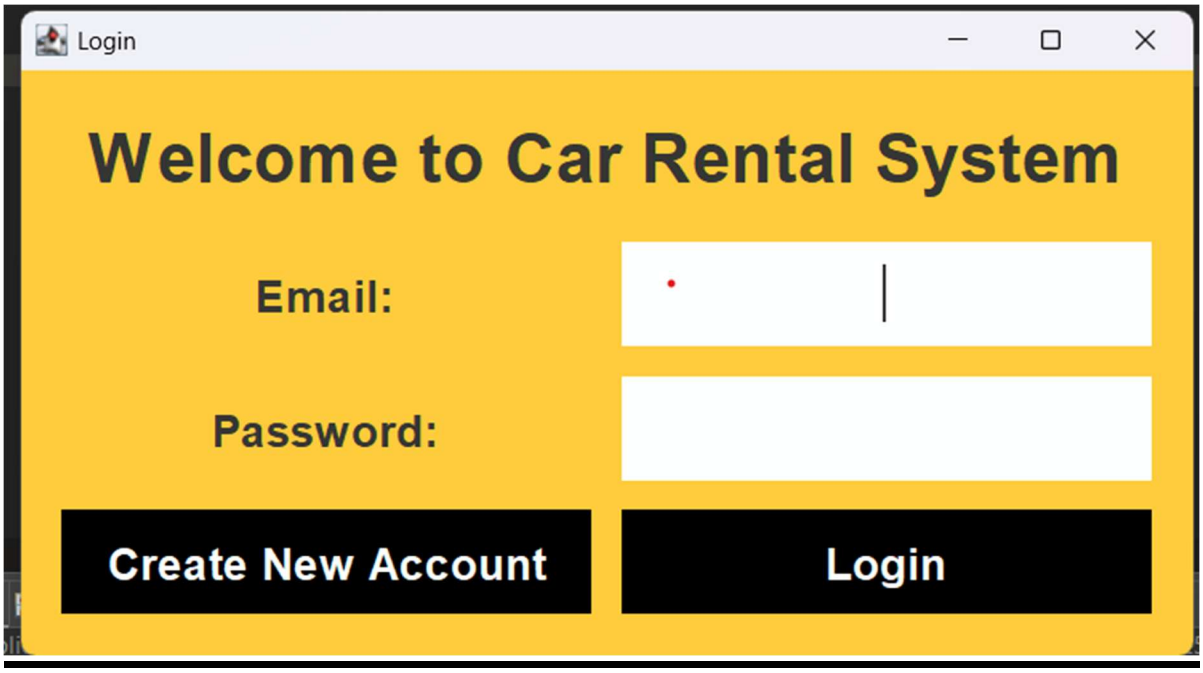
```
83      -- users with no rentals
84  •   SELECT u.ID, u.FirstName, u.Email
85      FROM users u
86      LEFT JOIN rents r ON u.ID = r.User
87      WHERE r.ID IS NULL;
88
```

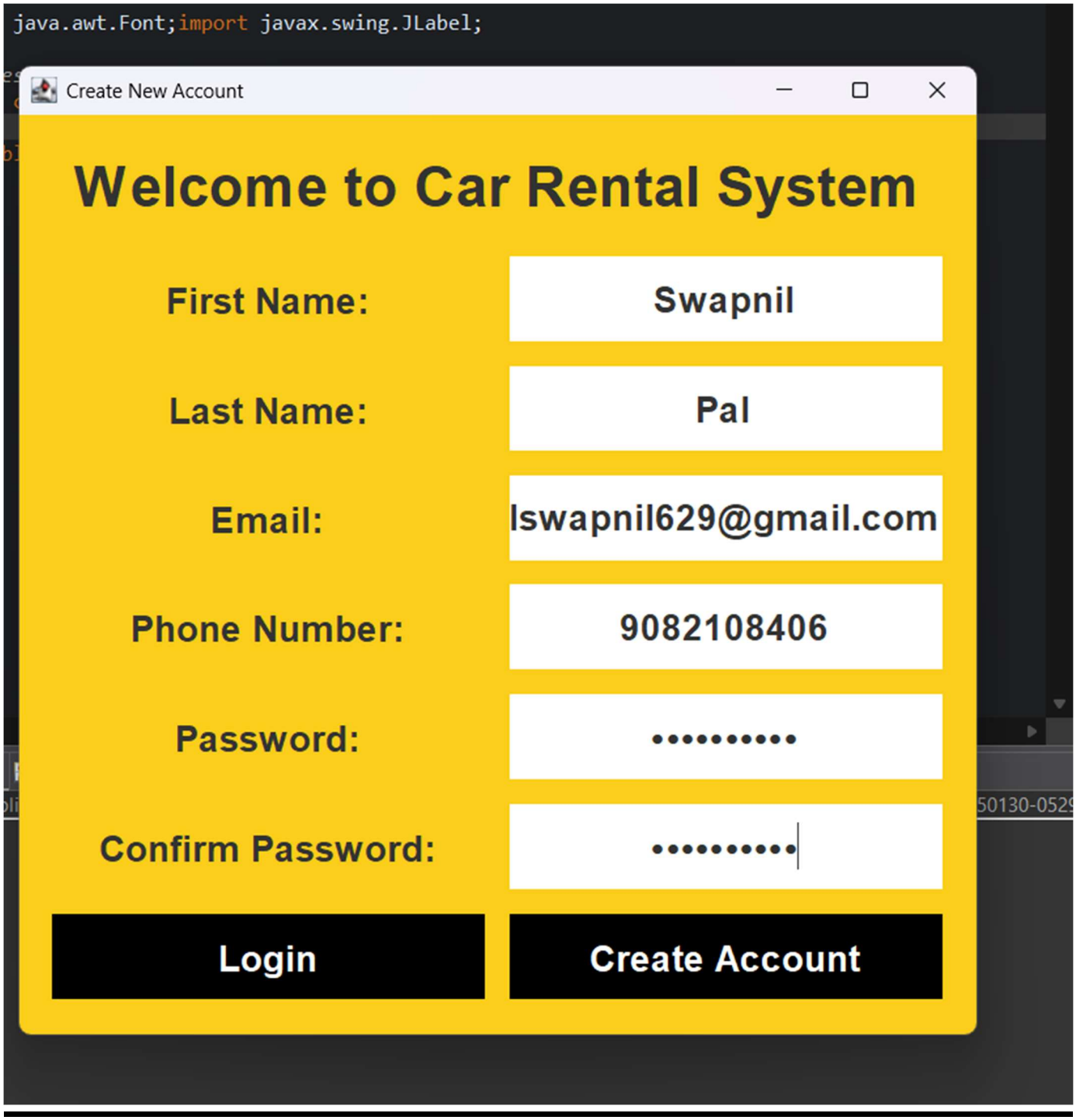
Result Grid | Filter Rows: | Export:

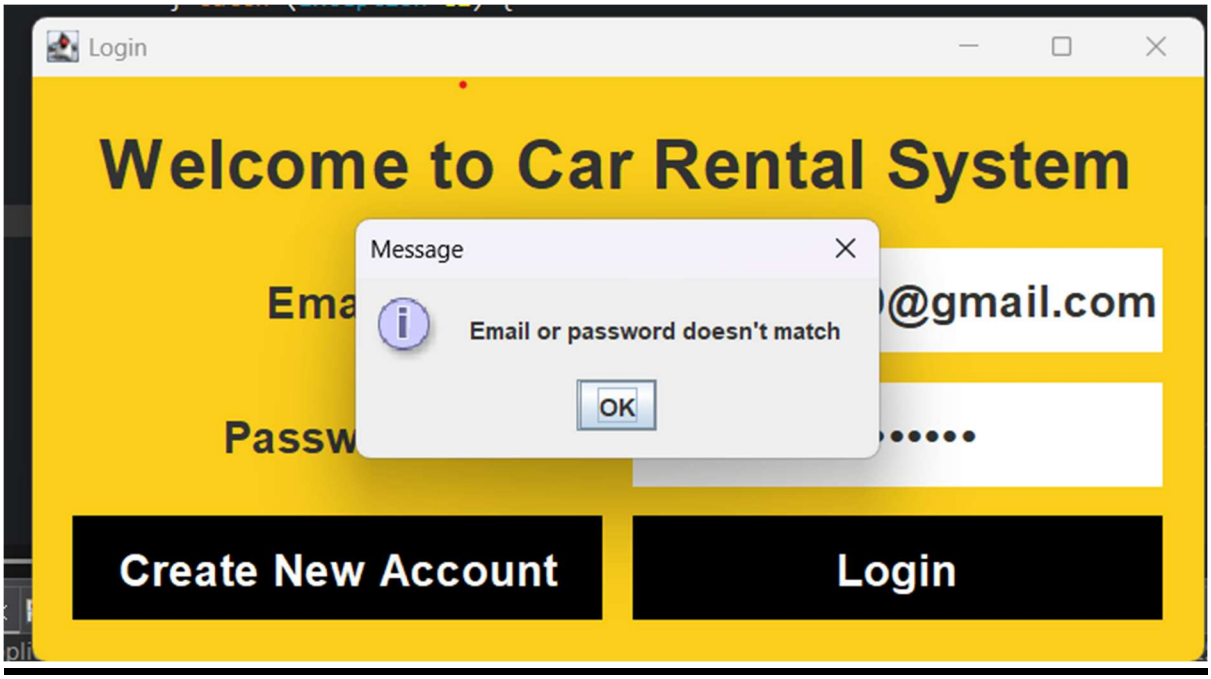
ID	FirstName	Email
1	Admin	admin
3	Client	client1
4	Client	client2@crs.com
5	Client	client3@crs.com
6	Client	client4@crs.com

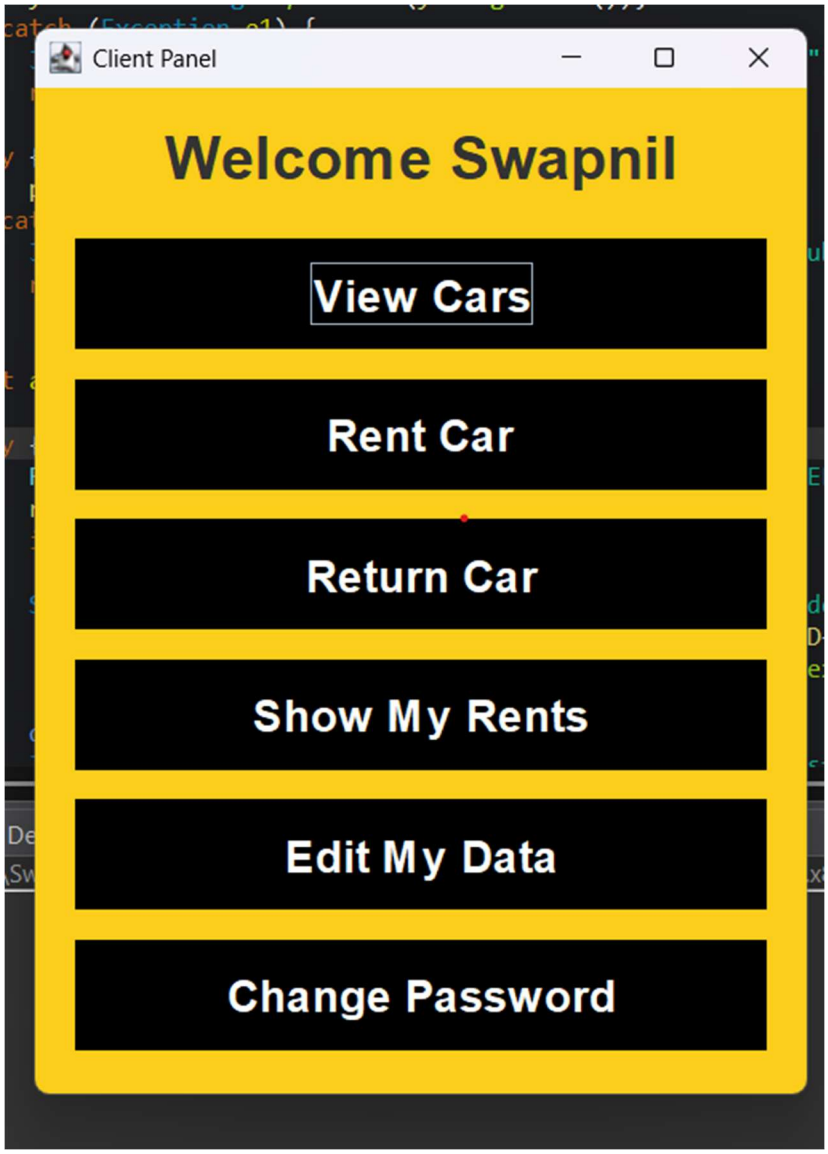
VII.PROJECT DEMONSTRATION

Tool / Software	Purpose
Eclipse IDE	Java code development and GUI design using Swing
Java (JDK 21)	Backend programming language; used for logic and JDBC connectivity
MySQL	Relational Database Management System used to store and manage data
phpMyAdmin	Web-based interface to create, modify, and view the MySQL database
XAMPP	Local server environment to run MySQL and phpMyAdmin locally
MySQL Workbench	SQL query development, ER modeling, and execution with joins/analytics
JDBC (Java Connectivity)	Connects Java application to MySQL database
Swing (Java GUI Library)	Used to build the graphical interface for login, car management, etc.


Login
Welcome to Car Rental System
Email:
Password:
Create New Account
Login

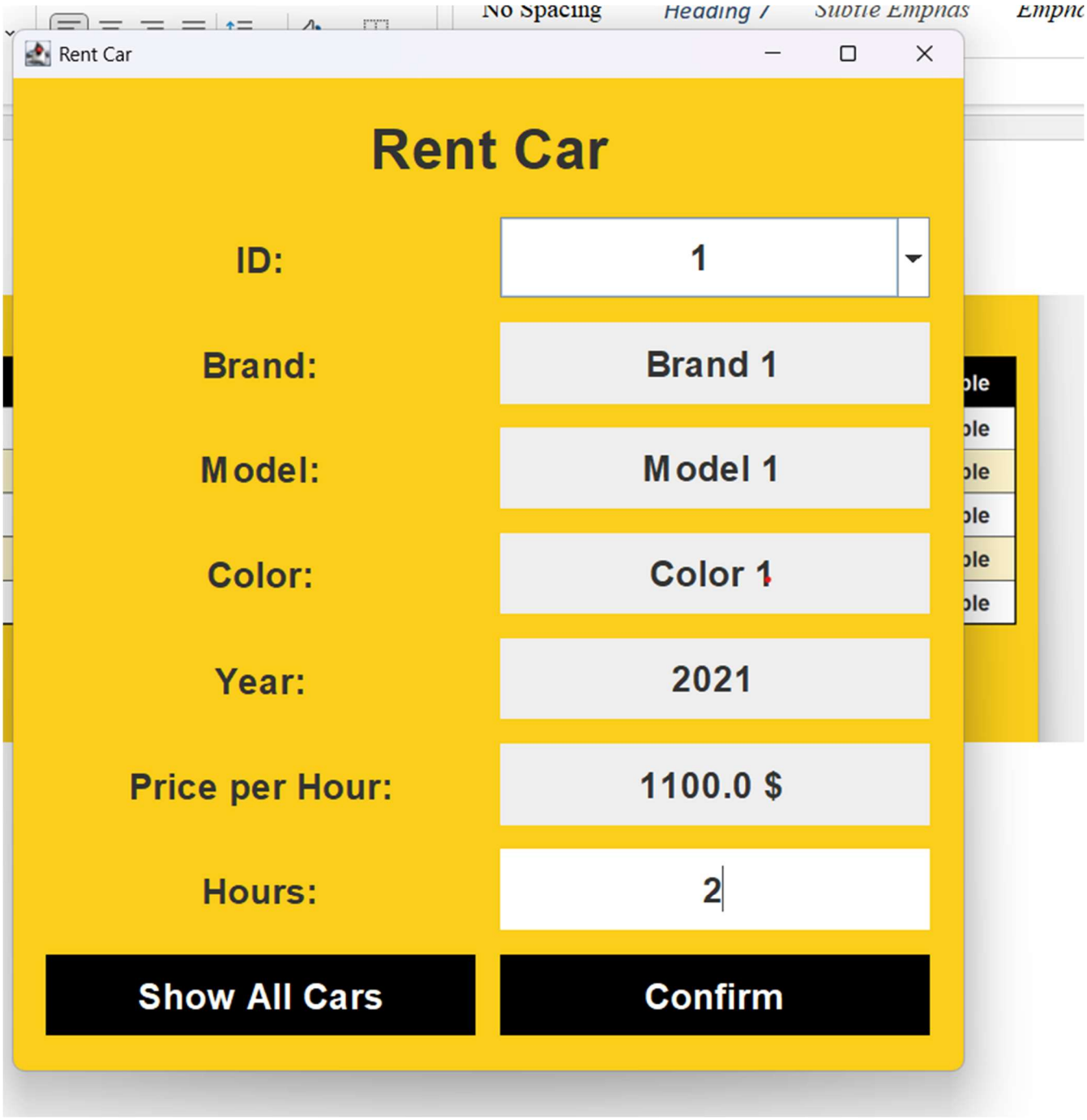

java.awt.Font;import javax.swing.JLabel;
Create New Account
Welcome to Car Rental System
First Name:
Swapnil
Last Name:
Pal
Email:
lswapnil629@gmail.com
Phone Number:
9082108406
Password:
Confirm Password:
Login
Create Account

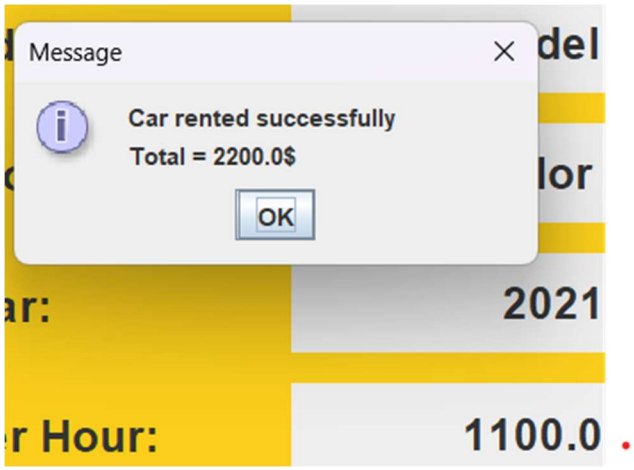

Login
Welcome to Car Rental System
Message
Email or password doesn't match
OK
@gmail.com
Create New Account
Login


Client Panel
Welcome Swapnil
View Cars
Rent Car
Return Car
Show My Rents
Edit My Data
Change Password

Cars

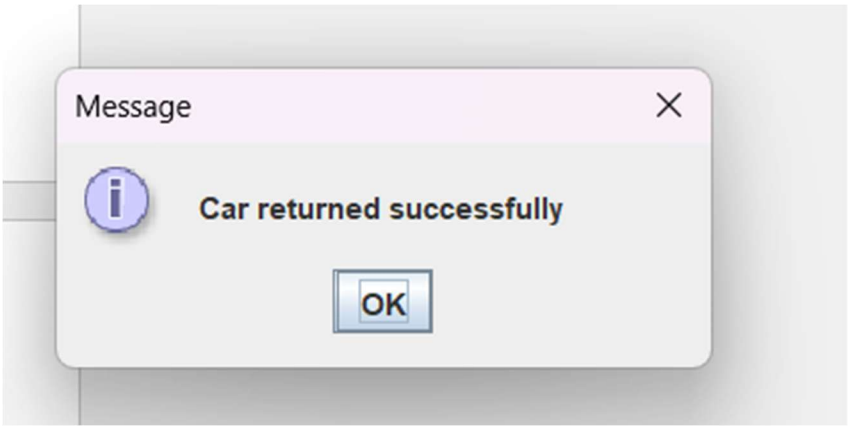
ID	Brand	Model	Color	Year	Price	Available
0	Brand 0	Model 0	Color 0	2020	1000.0 $	Available
1	Brand 1	Model 1	Color 1	2021	1100.0 $	Available
2	Brand 2	Model 2	Color 2	2022	1200.0 $	Available
3	Brand 3	Model 3	Color 3	2023	1300.0 $	Available
4	Brand 4	Model 4	Color 4	2024	1400.0 $	Available

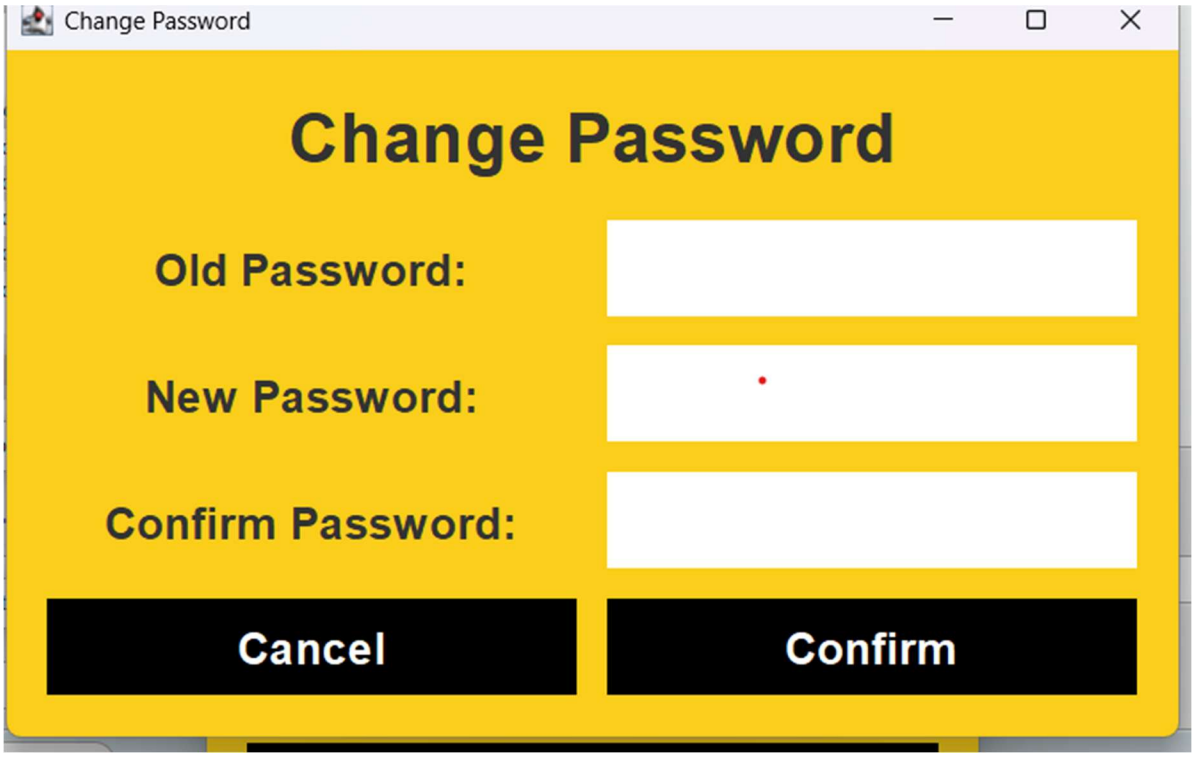

No Spacing
Heading 1
Subtle Emphas
Rent Car
Rent Car
ID:
1
Brand:
Brand 1
Model:
Model 1
Color:
Color 1
Year:
2021
Price per Hour:
1100.0 $
Hours:
2
Show All Cars
Confirm



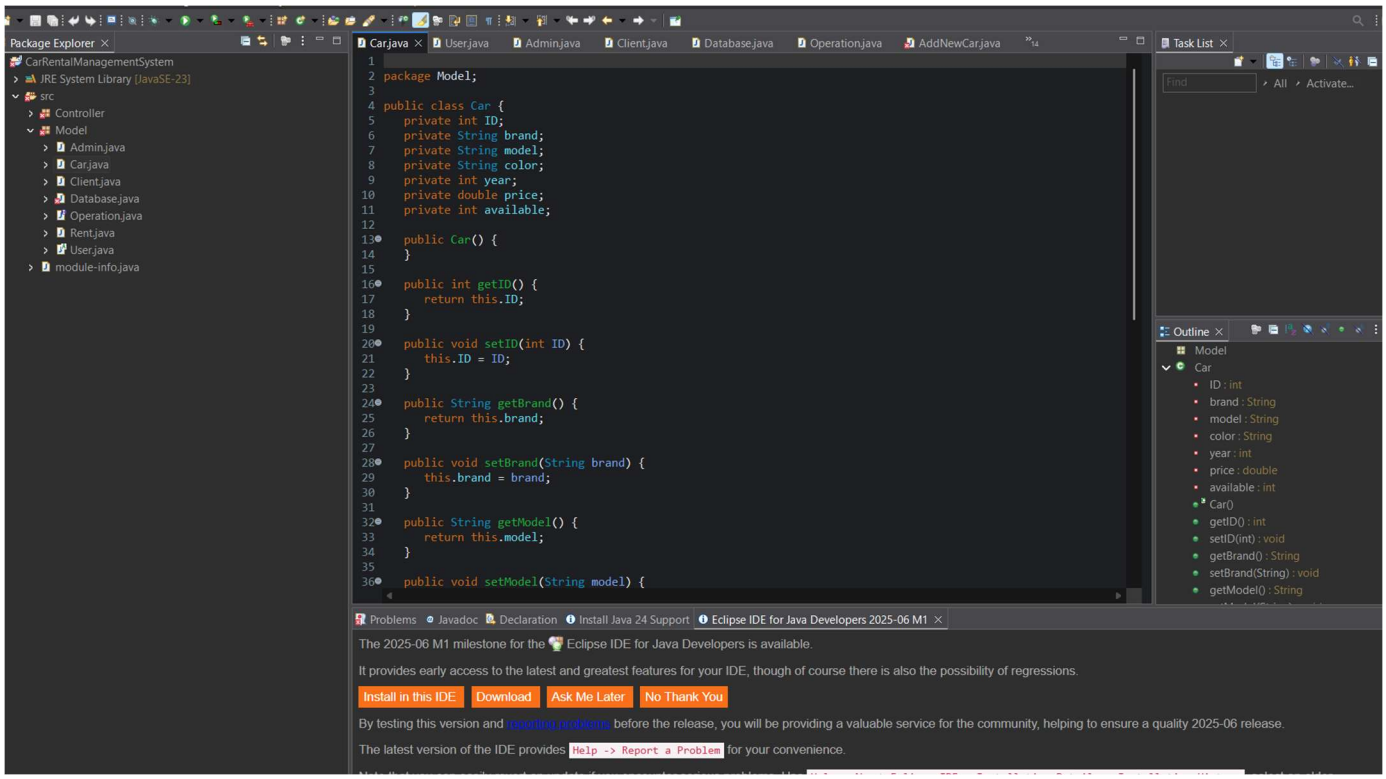
Rents

ID	Name	Email	Tel	Car ID	Car	Date Time	Hours	Total	Statu
9	Swapnil Pal	palswapn...	90821084...	1	Brand 1 ...	2025-14-0...	2	2200.0 $	Estimat

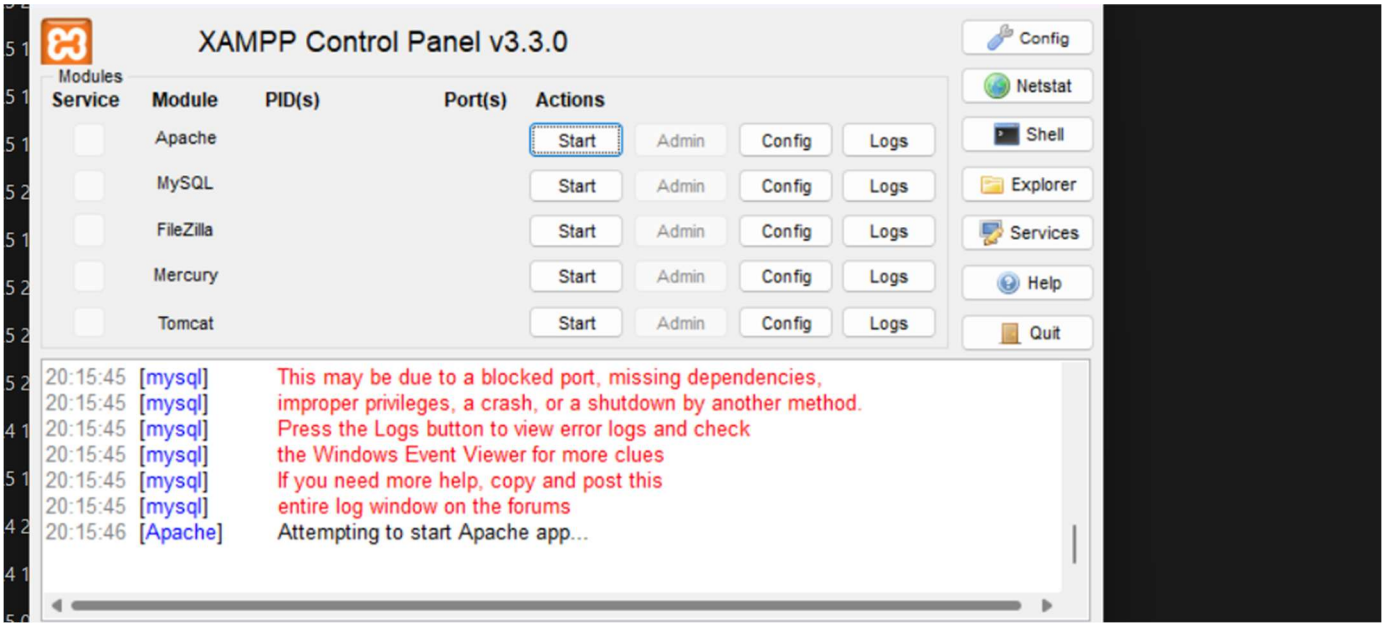

Message
Car returned successfully
OK


Change Password
Change Password
Old Password:
New Password:
Confirm Password:
Cancel
Confirm

IMPLEMENTATION OF THE PROJECT:




apachefriends.org/index.html
Apache Friends
Download
Hosting
Community
About
Search..
Search
EN
XAMPP Apache + MariaDB + PHP + Perl
What is XAMPP?
XAMPP is the most popular PHP development environment
XAMPP is a completely free, easy to install Apache distribution containing MariaDB, PHP, and Perl. The XAMPP open source package has been set up to be incredibly easy to install and to use.
XAMPP
Download
Click here for other versions
XAMPP for Windows
8.2.12 (PHP 8.2.12)
XAMPP for Linux
8.2.12 (PHP 8.2.12)
XAMPP for OS X
8.2.4 (PHP 8.2.4)
New XAMPP release 8.2.12,
Hi Apache Friends!
We just released a new version of XAMPP for Windows for PHP versions 8.2.12, 8.1.25 and 8.0.30. New versions for Linux and OS X will come soon! You can

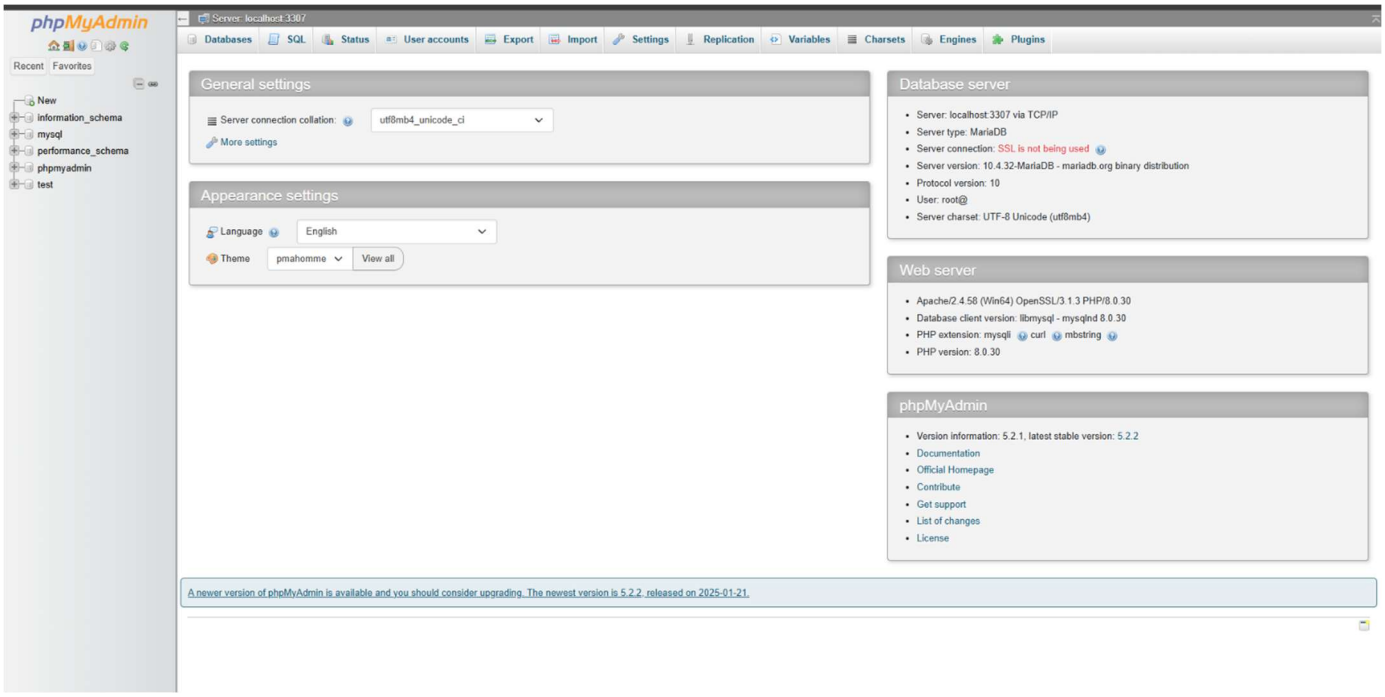

XAMPP Control Panel v3.3.0
Modules
Service
Module
PID(s)
Port(s)
Actions
Apache
MySQL
FileZilla
Mercury
Tomcat
Start
Admin
Config
Logs
Config
Netstat
Shell
Explorer
Services
Help
Quit
20:15:45 [mysql] This may be due to a blocked port, missing dependencies,
20:15:45 [mysql] improper privileges, a crash, or a shutdown by another method.
20:15:45 [mysql] Press the Logs button to view error logs and check
20:15:45 [mysql] the Windows Event Viewer for more clues
20:15:45 [mysql] If you need more help, copy and post this
20:15:45 [mysql] entire log window on the forums
20:15:46 [Apache] Attempting to start Apache app...

```
Setting environment for using XAMPP for Windows.
Swapnil Pal@LAPTOP-UDTGUD0Q c:\Users\Swapnil Pal\Desktop\XAMPP
# netstat -aon | findstr :80
  TCP    192.168.29.125:50171   192.168.29.162:8009    ESTABLISHED     25564

Swapnil Pal@LAPTOP-UDTGUD0Q c:\Users\Swapnil Pal\Desktop\XAMPP
# netstat -aon | findstr :443
  TCP    192.168.29.125:50187   3.111.224.186:443      ESTABLISHED     5780
  TCP    192.168.29.125:50209   23.212.0.108:443       ESTABLISHED     16904
  TCP    192.168.29.125:50230   198.41.30.198:443      CLOSE_WAIT      25084
  TCP    192.168.29.125:51457   184.86.248.146:443     ESTABLISHED     25564
  TCP    [2405:201:3e:a0cb:b490:86ac:5188:7659]:49408  [2603:1040:a06:6::]:443  ESTABLISHED     9092
  TCP    [2405:201:3e:a0cb:b490:86ac:5188:7659]:50022  [2603:1063:c::6c]:443  ESTABLISHED     11900
  TCP    [2405:201:3e:a0cb:b490:86ac:5188:7659]:50050  [2603:1063:15::102]:443  ESTABLISHED     21788
  TCP    [2405:201:3e:a0cb:b490:86ac:5188:7659]:51404  [2603:1046:900:2c::2]:443  ESTABLISHED     10704
  TCP    [2405:201:3e:a0cb:b490:86ac:5188:7659]:51463  [2405:200:1602:2885:face:b00c:3333:7020]:443  CLOSE_WAIT      265
56
  TCP    [2405:201:3e:a0cb:b490:86ac:5188:7659]:51468  [2606:4700:8d72:7cbc:c2ca:6e:5ff2:75c4]:443  ESTABLISHED     9748
  UDP    0.0.0.0:62915          49.44.130.62:443                   18856
  UDP    [::]:62915             [2405:200:1602::312c:823e]:443                 18856

Swapnil Pal@LAPTOP-UDTGUD0Q c:\Users\Swapnil Pal\Desktop\XAMPP
# S
```

ChatGPT can make mistakes. Check important info.



VIII. SELF LEARNING BEYOND THE CLASSROOM

This project required exploring technologies and tools that extended far beyond the classroom curriculum. While classroom sessions typically focus on theoretical knowledge of DBMS concepts and basic SQL, this project encouraged deep dives into:

- **phpMyAdmin usage:** While MySQL was taught, GUI-based operations in phpMyAdmin like table creation, relationship setup, indexing, and executing raw SQL required individual experimentation.
- **MySQL Workbench for query visualization:** Beyond phpMyAdmin, Workbench helped in writing more complex join queries and understanding the schema at a broader level.
- **Java GUI development using Swing:** Implementing front-end interfaces like login forms, account creation, and rental operations using Java Swing was largely self-taught.
- **JDBC Integration:** Understanding how Java connects to a database via JDBC wasn't just about copying code; it involved learning driver loading, connection handling, statement execution, and result parsing.

IX. LEARNING FROM THE PROJECT

This project brought together various DBMS and programming concepts into one real-life use case. Key learnings include:

- **Database Design**: Designing a clean and normalized schema with proper keys, constraints, and relationships for entities like users, cars, and rents.
- **CRUD Operations**: Implementing Create, Read, Update, and Delete functionality via both SQL and Java code gave a practical understanding of data manipulation.
- **Query Writing and Joins**: Gained experience writing SELECT, UPDATE, and DELETE queries, especially with **INNER and OUTER JOINS** to connect multiple tables and retrieve combined data (e.g., user rent history).
- **Java Exception Handling**: Dealing with common runtime exceptions such as SQLException, NullPointerException, and connection issues taught the importance of robust error management.
- **User Management System**: Designing different access levels for admins and clients deepened understanding of role-based access control in databases and applications.
- **GUI Integration**: Creating a responsive and user-friendly interface using Java Swing improved front-end logic design and event handling.

X. CHALLENGES FACED

```
1  INSERT INTO rents (ID, User, Car, DateTime, Hours, Total, Status) VALUES
2  (0, 2, 7, '2023-22-12 23:59', 2, 3400, 0),
3  (1, 2, 0, '2023-22-12 23:59', 7, 7000, 0),
4  (2, 2, 2, '2023-23-12 00:00', 3, 3600, 0),
5  (3, 2, 3, '2023-23-12 00:16', 1, 1300, 0),
6  (4, 2, 5, '2023-23-12 00:16', 2, 3000, 0),
7  (5, 2, 5, '2023-23-12 00:16', 5, 7500, 0),
8  (6, 2, 9, '2023-23-12 00:16', 8, 14400, 0),
9  (7, 2, 7, '2023-23-12 00:16', 7, 11900, 0),
10 (8, 2, 5, '2023-23-12 00:16', 1, 1500, 0);
```

SELECT * | SELECT | INSERT | UPDATE | DELETE | Clear | Format | Get auto-saved query

Bind parameters

Bookmark this SQL query:

Delimiter ; | Show this query here again | Retain query box | Rollback when finished | Enable foreign key checks | Go

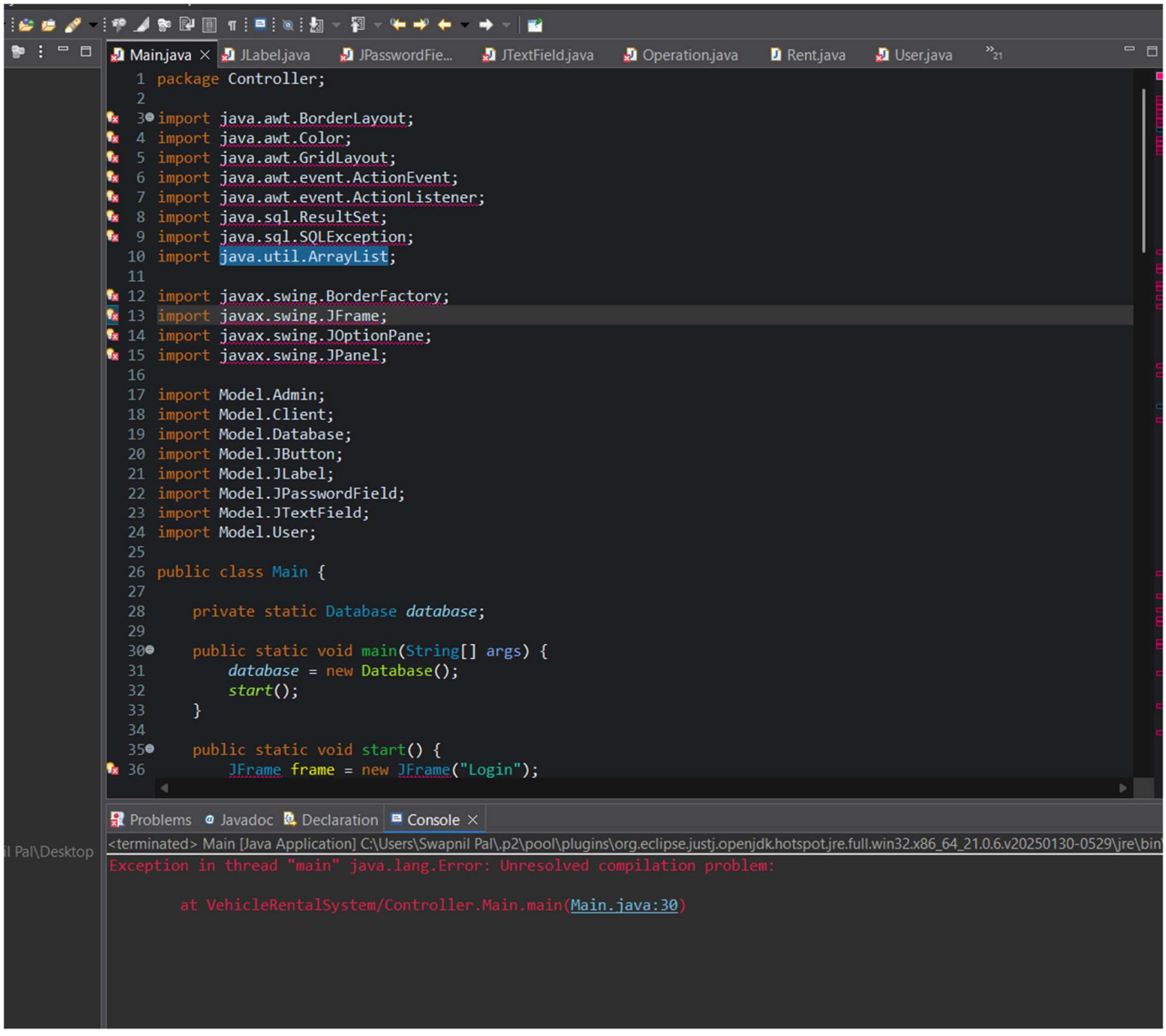
Error

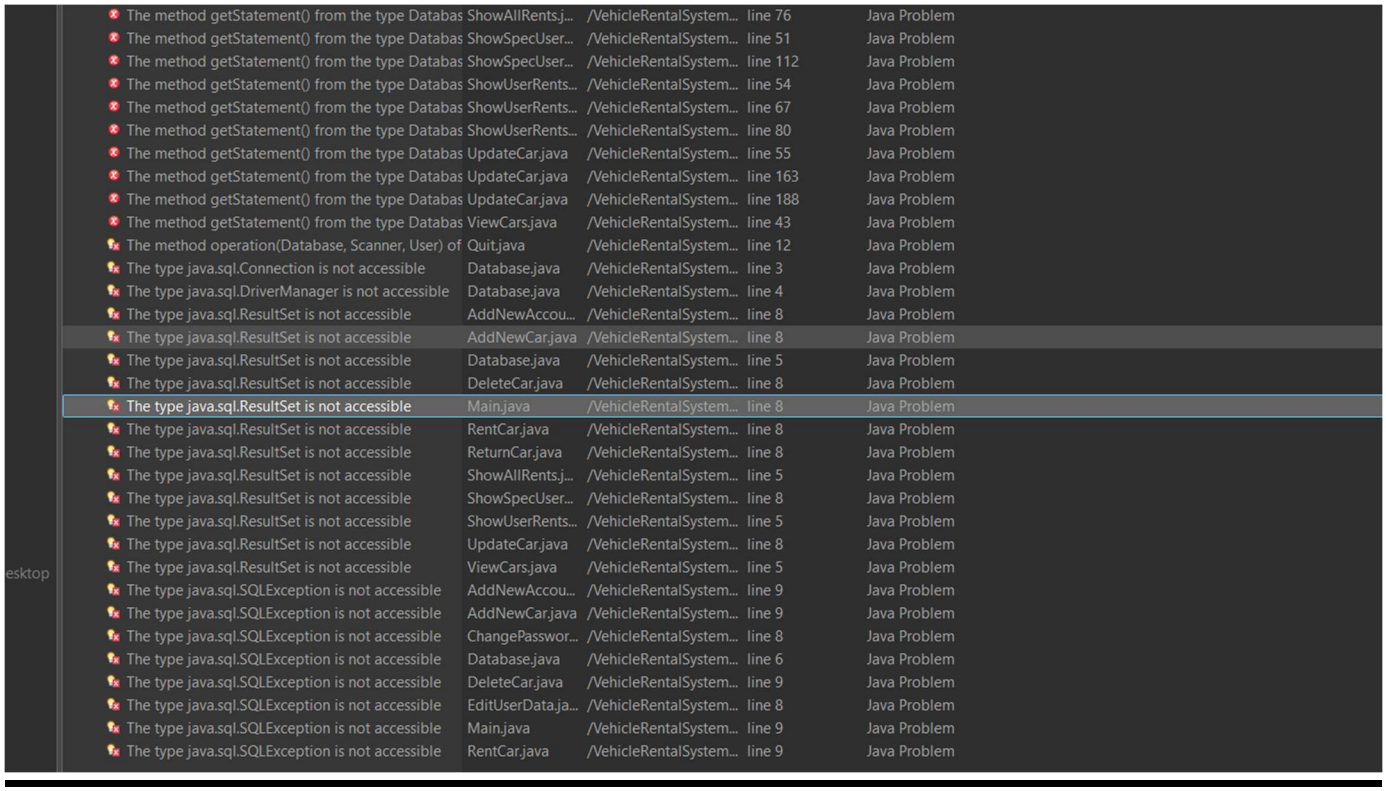
SQL query: Copy

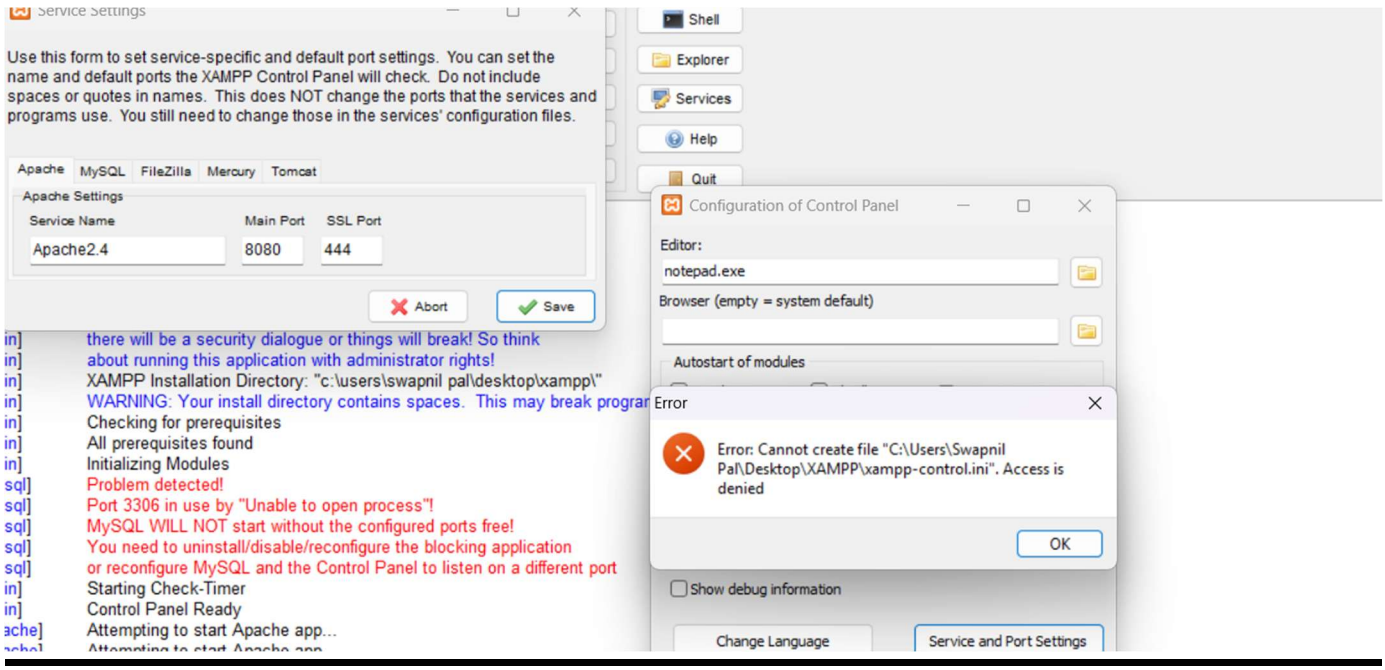
```
INSERT INTO rents (ID, User, Car, DateTime, Hours, Total, Status) VALUES
(0, 2, 7, '2023-22-12 23:59', 2, 3400, 0),
(1, 2, 0, '2023-22-12 23:59', 7, 7000, 0),
(2, 2, 2, '2023-23-12 00:00', 3, 3600, 0),
(3, 2, 3, '2023-23-12 00:16', 1, 1300, 0),
(4, 2, 5, '2023-23-12 00:16', 2, 3000, 0),
```

MySQL said:

```
1054 - Unknown column ' 1' in 'field list'
```


Main.java
JLabel.java
JPasswordFie...
JTextField.java
Operation.java
Rent.java
User.java
1 package Controller;
2
3 import java.awt.BorderLayout;
4 import java.awt.Color;
5 import java.awt.GridLayout;
6 import java.awt.event.ActionEvent;
7 import java.awt.event.ActionListener;
8 import java.sql.ResultSet;
9 import java.sql.SQLException;
10 import java.util.ArrayList;
11
12 import javax.swing.BorderFactory;
13 import javax.swing.JFrame;
14 import javax.swing.JOptionPane;
15 import javax.swing.JPanel;
16
17 import Model.Admin;
18 import Model.Client;
19 import Model.Database;
20 import Model.JButton;
21 import Model.JLabel;
22 import Model.JPasswordField;
23 import Model.JTextField;
24 import Model.User;
25
26 public class Main {
27
28 private static Database database;
29
30 public static void main(String[] args) {
31 database = new Database();
32 start();
33 }
34
35 public static void start() {
36 JFrame frame = new JFrame("Login");
Problems
Javadoc
Declaration
Console
<terminated> Main [Java Application] C:\Users\Swapnil Pal\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.6.v20250130-0529\jre\bin
Exception in thread "main" java.lang.Error: Unresolved compilation problem:
at VehicleRentalSystem/Controller.Main.main(Main.java:30)


The method getStatement() from the type Databas ShowAllRents.j... /VehicleRentalSystem... line 76 Java Problem
The method getStatement() from the type Databas ShowSpecUser... /VehicleRentalSystem... line 51 Java Problem
The method getStatement() from the type Databas ShowSpecUser... /VehicleRentalSystem... line 112 Java Problem
The method getStatement() from the type Databas ShowUserRents... /VehicleRentalSystem... line 54 Java Problem
The method getStatement() from the type Databas ShowUserRents... /VehicleRentalSystem... line 67 Java Problem
The method getStatement() from the type Databas ShowUserRents... /VehicleRentalSystem... line 80 Java Problem
The method getStatement() from the type Databas UpdateCar.java /VehicleRentalSystem... line 55 Java Problem
The method getStatement() from the type Databas UpdateCar.java /VehicleRentalSystem... line 163 Java Problem
The method getStatement() from the type Databas UpdateCar.java /VehicleRentalSystem... line 188 Java Problem
The method getStatement() from the type Databas ViewCars.java /VehicleRentalSystem... line 43 Java Problem
The method operation(Database, Scanner, User) of Quit.java /VehicleRentalSystem... line 12 Java Problem
The type java.sql.Connection is not accessible Database.java /VehicleRentalSystem... line 3 Java Problem
The type java.sql.DriverManager is not accessible Database.java /VehicleRentalSystem... line 4 Java Problem
The type java.sql.ResultSet is not accessible AddNewAccou... /VehicleRentalSystem... line 8 Java Problem
The type java.sql.ResultSet is not accessible AddNewCar.java /VehicleRentalSystem... line 8 Java Problem
The type java.sql.ResultSet is not accessible Database.java /VehicleRentalSystem... line 5 Java Problem
The type java.sql.ResultSet is not accessible DeleteCar.java /VehicleRentalSystem... line 8 Java Problem
The type java.sql.ResultSet is not accessible Main.java /VehicleRentalSystem... line 8 Java Problem
The type java.sql.ResultSet is not accessible RentCar.java /VehicleRentalSystem... line 8 Java Problem
The type java.sql.ResultSet is not accessible ReturnCar.java /VehicleRentalSystem... line 8 Java Problem
The type java.sql.ResultSet is not accessible ShowAllRents.j... /VehicleRentalSystem... line 5 Java Problem
The type java.sql.ResultSet is not accessible ShowSpecUser... /VehicleRentalSystem... line 8 Java Problem
The type java.sql.ResultSet is not accessible ShowUserRents... /VehicleRentalSystem... line 5 Java Problem
The type java.sql.ResultSet is not accessible UpdateCar.java /VehicleRentalSystem... line 8 Java Problem
The type java.sql.ResultSet is not accessible ViewCars.java /VehicleRentalSystem... line 5 Java Problem
The type java.sql.SQLException is not accessible AddNewAccou... /VehicleRentalSystem... line 9 Java Problem
The type java.sql.SQLException is not accessible AddNewCar.java /VehicleRentalSystem... line 9 Java Problem
The type java.sql.SQLException is not accessible ChangePasswor... /VehicleRentalSystem... line 8 Java Problem
The type java.sql.SQLException is not accessible Database.java /VehicleRentalSystem... line 6 Java Problem
The type java.sql.SQLException is not accessible DeleteCar.java /VehicleRentalSystem... line 9 Java Problem
The type java.sql.SQLException is not accessible EditUserData.ja... /VehicleRentalSystem... line 8 Java Problem
The type java.sql.SQLException is not accessible Main.java /VehicleRentalSystem... line 9 Java Problem
The type java.sql.SQLException is not accessible RentCar.java /VehicleRentalSystem... line 9 Java Problem


Service Settings
Use this form to set service-specific and default port settings. You can set the name and default ports the XAMPP Control Panel will check. Do not include spaces or quotes in names. This does NOT change the ports that the services and programs use. You still need to change those in the services' configuration files.
Apache MySQL FileZilla Mercury Tomcat
Apache Settings
Service Name Main Port SSL Port
Apache2.4 8080 444
Abort Save
Shell
Explorer
Services
Help
Quit
Configuration of Control Panel
Editor:
notepad.exe
Browser (empty = system default)
Autostart of modules
Error
Error: Cannot create file "C:\Users\Swapnil Pal\Desktop\XAMPP\xampp-control.ini". Access is denied
OK
Show debug information
Change Language
Service and Port Settings
there will be a security dialogue or things will break! So think
about running this application with administrator rights!
XAMPP Installation Directory: "c:\users\swapnil pal\desktop\xampp\"
WARNING: Your install directory contains spaces. This may break prograr
Checking for prerequisites
All prerequisites found
Initializing Modules
Problem detected!
Port 3306 in use by "Unable to open process"!
MySQL WILL NOT start without the configured ports free!
You need to uninstall/disable/reconfigure the blocking application
or reconfigure MySQL and the Control Panel to listen on a different port
Starting Check-Timer
Control Panel Ready
Attempting to start Apache app...

Throughout the development of the Car Rental System project, we encountered several real-world challenges that tested our understanding and problem-solving skills beyond the textbook. One of the earliest issues was related to database connectivity between Java and MySQL. Even though we included the JDBC connector, the connection failed multiple times due to incorrect port configuration and credential mismatches. This led us to understand the importance of checking default MySQL port settings (in our case, port 3307 on XAMPP) and ensuring consistency between phpMyAdmin and Eclipse JDBC setup.

Another frequent issue was SQL exceptions thrown during runtime. Often, these occurred due to syntactical mistakes, missing primary keys, or violations of foreign key constraints. For example, while inserting data into the rents table, we received duplicate entry errors for the ID field because it wasn't auto-incremented. Additionally, referencing foreign keys that didn't exist yet caused failures until we learned to insert dependent data first or adjust the schema accordingly.

Working with Swing GUI in Java presented its own set of complexities. Laying out buttons, labels, and input fields in a visually clean manner was difficult without understanding layout managers. At times, our application windows appeared misaligned or controls overlapped, so we had to explore layout strategies like GridLayout and BorderLayout to get it right. Also, handling button actions and triggering SQL queries from within the GUI involved learning about ActionListeners and managing input data effectively.

We also faced difficulty when switching to MySQL Workbench for writing advanced queries. Unlike phpMyAdmin, Workbench required us to explicitly set the default schema using the USE command, which we initially overlooked. This caused queries to fail with "no database selected" errors until we learned how to configure schemas properly. Complex INNER JOIN and LEFT JOIN queries required a clear understanding of relationships across users, cars, and rents tables, and we had to carefully design our queries to avoid ambiguous column references.

One of the most conceptually challenging aspects was ensuring the database adhered to normalization standards. While 1NF was relatively straightforward, applying 2NF and 3NF in a practical scenario required us to remove partial and transitive dependencies — especially when dealing with attributes like brand, model, and availability. We refined our relational model multiple times to ensure that no redundant or repeating data was present.

Lastly, working with user roles (Admin vs Client) in the system required thoughtful validation. Ensuring that each user could only access relevant features involved logic both in the database and the front-end Java interface. Building this role-based access control improved the system's security and professionalism but was only possible after careful design and debugging.

XI. CONCLUSION

The Vehicle Rental Management System DBMS project was not just a coding task about MY SQL queries and Java— it was a deep exploration of real-world application development. It helped us connect abstract database concepts with concrete implementation using tools like Java, JDBC, phpMyAdmin, and MySQL Workbench. From designing schemas to querying relational data and integrating with a functional Java interface, we walked through the complete development lifecycle.

This project helped us to sharpened our logical thinking, programming skills, database fluency, and presentation abilities. It prepared us not only for DBMS exams but also for future internships and industry-level full-stack development. Most importantly, it taught us how to learn independently, troubleshoot problems, and turn theory into impactful applications.